

MindSpore Transformers 大模型系列课程

MindSpore Transformers 训练高可用特性介绍

主讲人: HighA 昇思MindSpore高可用技术专家

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

🚩 《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

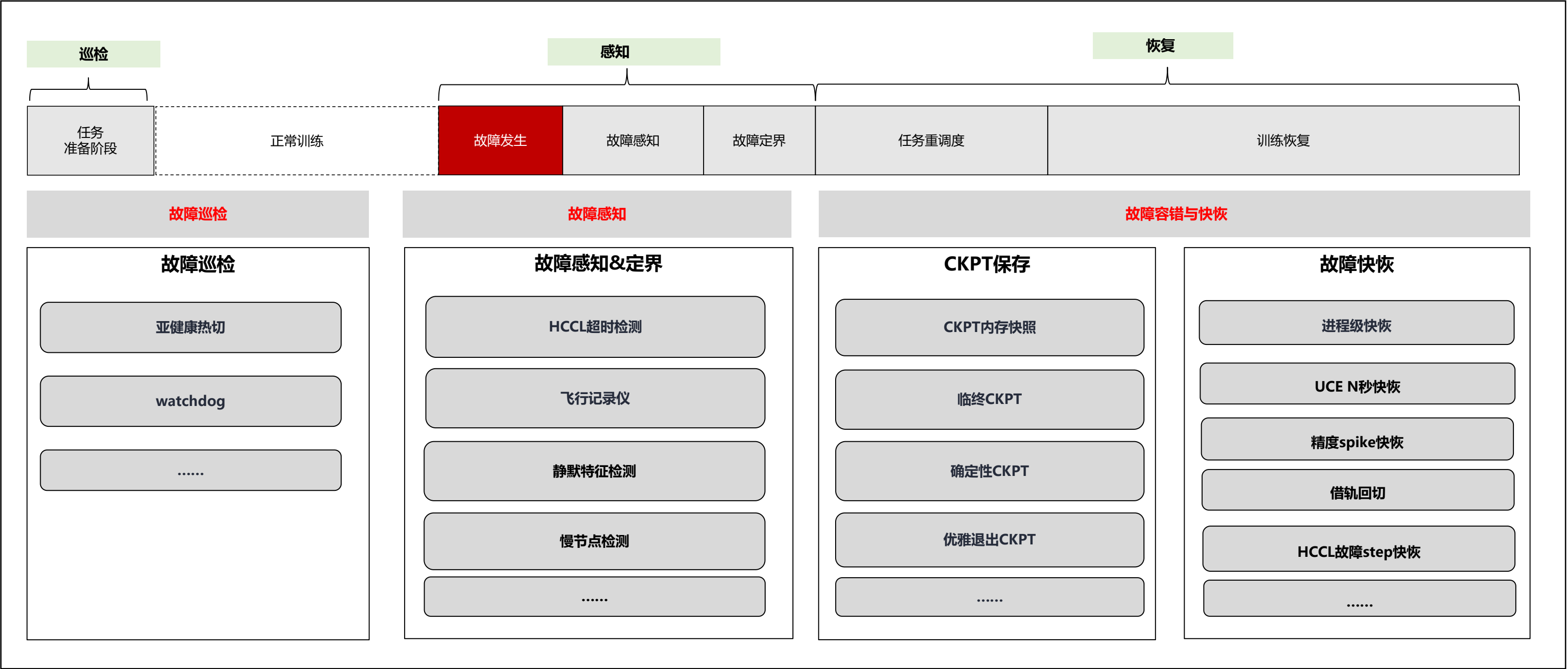
《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

《MindSpore Transformers DeepSeek-R1蒸馏实践》

MindSpore Transformers训练高可用



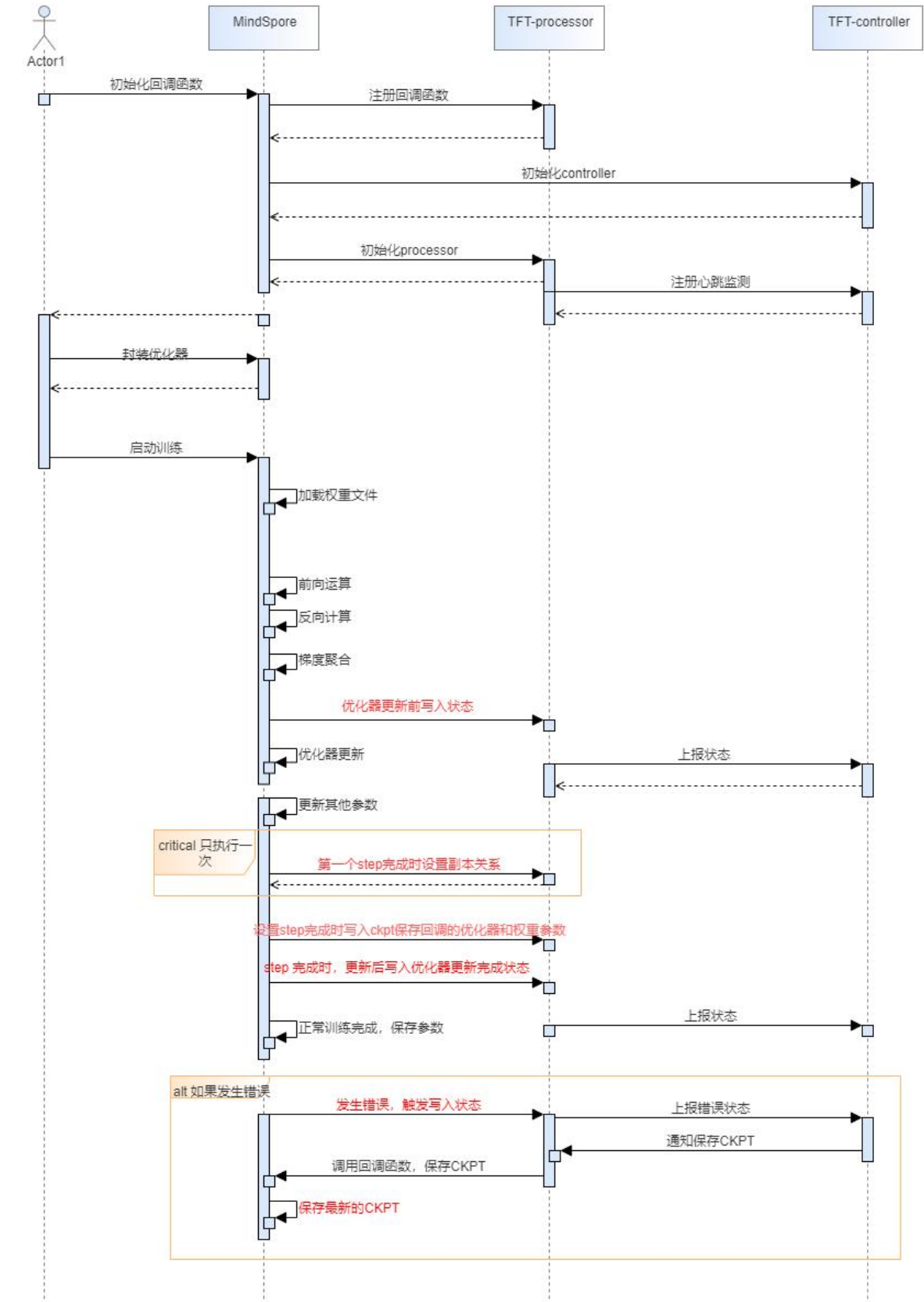
一、功能介绍:

临终遗言功能，该功能针对大模型训练过程中故障恢复加速，临终遗言特性通过在训练过程中发生故障后，校验中间状态数据的完整性和一致性，生成一次临时Checkpoint数据，恢复训练时能够通过该CheckPoint数据恢复，减少故障造成的训练迭代损失。

二、执行流程:

- 启动训练，注册ckpt保存回调函数
- 第一个step训练完成后，设置副本关系
- 优化器更新前写入未开始更新状态，优化器更新后写入完成状态
- 训练发生错误，触发写入状态，调回调函数保存。

三、使用方式：通过环境变量export MS_ENABLE_TFT={TTP:1}使能框架能力 TTP: Try To Persist



进程级快恢

一、功能介绍:

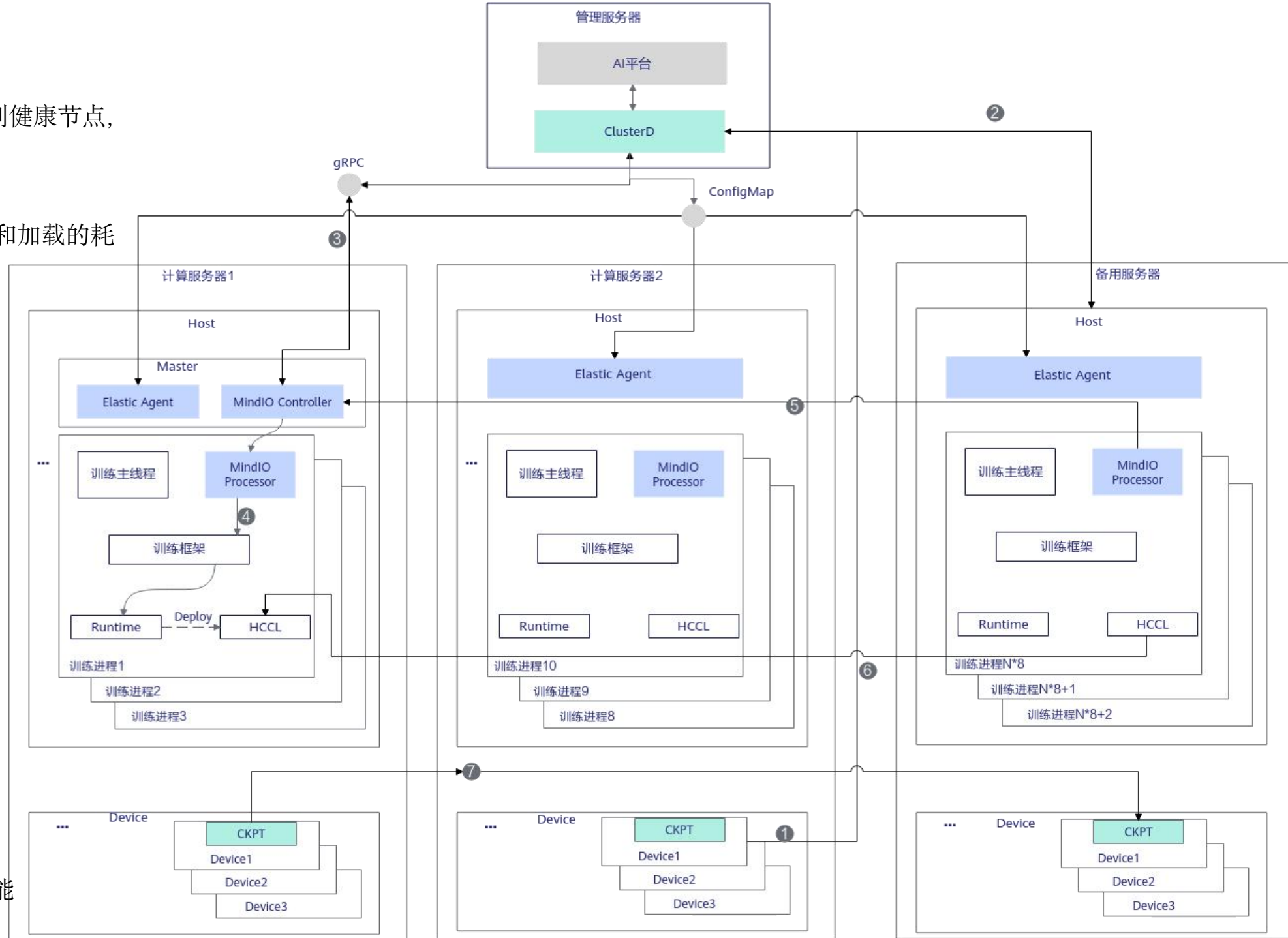
发生软硬件故障只停止故障相关节点的进程，然后将故障节点的容器迁移到健康节点，恢复训练任务。

1. 本功能仅重调度故障进程，减少了大量进程间不同步的等待耗时
2. 上层只需调度备用节点，无需调度整个集群，降低了建链耗时
3. 通过NPU卡间的参数面高速网络P2P传递CKPT信息，避免了CKPT保存和加载的耗时

二、执行流程:

- 设备出现硬件故障后，MindCluster在服务器上的检测组件上报故障信息到集群大脑中。
- ClusterD将故障服务器上的任务容器销毁，重新调度到备用的服务器上。
- ClusterD通知Master节点上的MindIO Controller进行容错。
- MindIO Controller通知每个训练进程中的MindIO Processor，通信停止。MindIO Processor清理正常节点的资源，销毁通信域，清理后等待新进程加入。
- 备用服务器上的管理进程拉起训练进程后，创建新的MindIO Processor，MindIO Controller通知每个训练进程中的MindIO Processor恢复训练。
- 各个进程进行集合通信建链。
- 正常服务器上的NPU通过参数面将CKPT传递到备用服务器上，完成参数状态恢复后继续训练。

三、使用方式：通过环境变量export MS_ENABLE_TFT={ARF:1}使能框架能力ARF: Air Refuelling



一、功能介绍：模型训练过程中，出现精度尖刺（loss或者梯度norm），继续训练会对后续训练精度产生影响，因此可通过进程不退出，回退到尖刺点之前重新训练，解决建此问题。

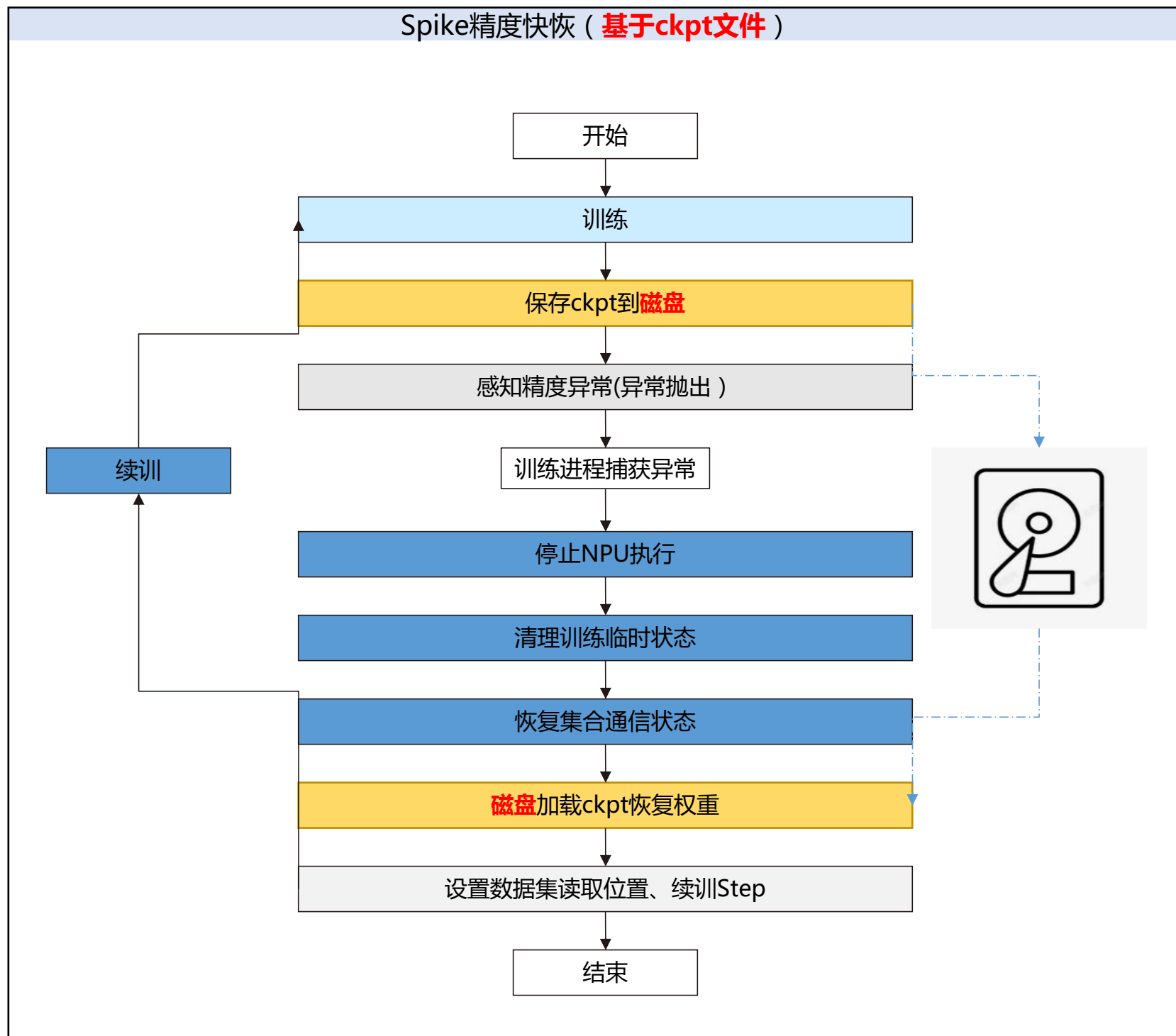
1. 进程不退出，无需重新创建通信域、图编译等耗时流程
2. 无需重新调度训练节点，减少集群调度和管理层面耗时

二、执行流程：

- MindSpore在训练的过程中当感知到精度或global norm值异常后，抛出 TREEError(Training Result Error) 异常
- MindSpore外侧循环体内捕获 TREEError 异常，使能精度故障不重启。
- 捕获TREEError异常后，执行以下动作进行状态清理恢复：
 - 停止设备
 - 清理运行时状态
 - 恢复hccl通信
 - 加载CheckPoint
 - 恢复训练

三、使用方式：通过环境变量export MS_ENABLE_TFT={TRE:1}使能框架能力

TRE: Training Result Error



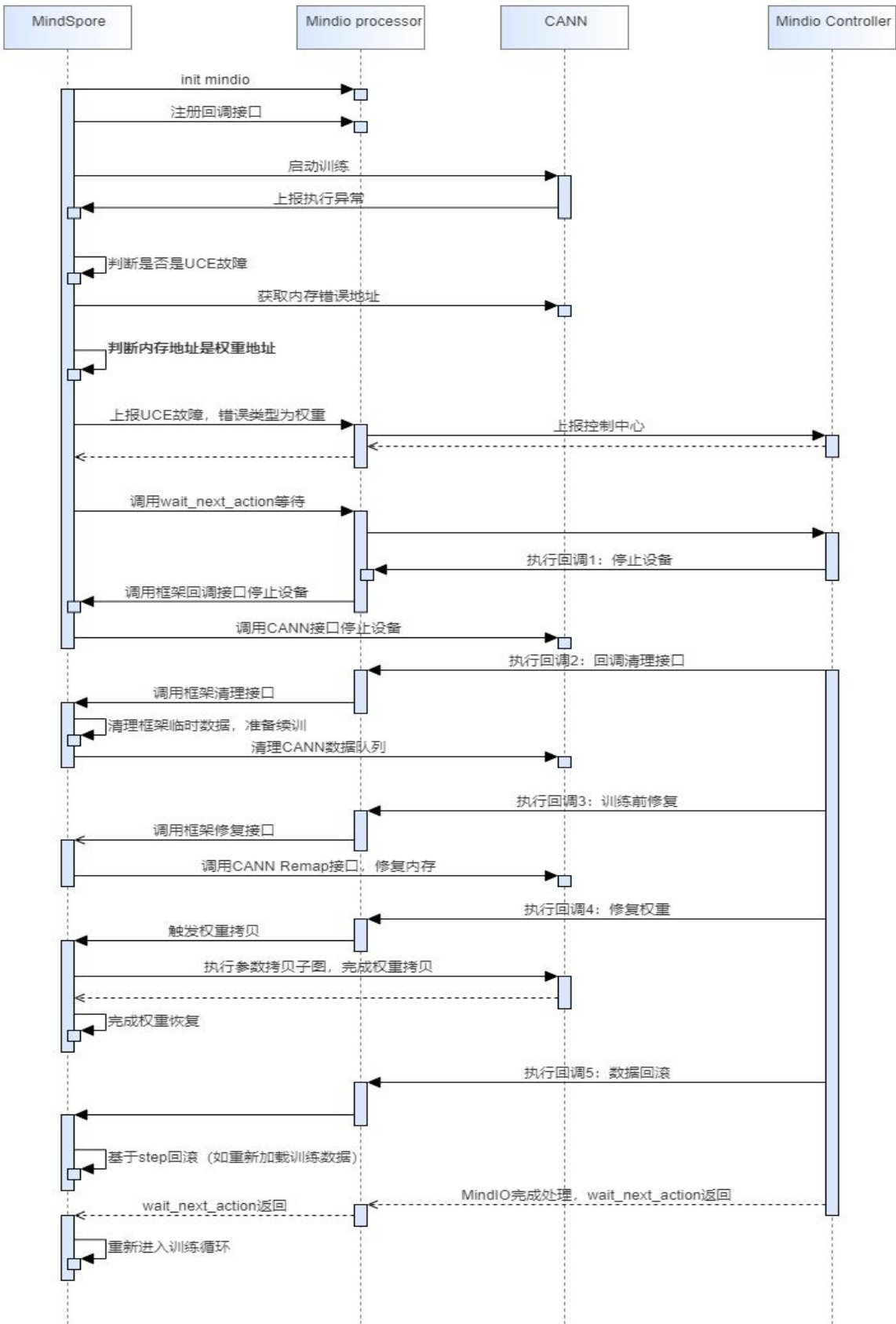
一、功能介绍：昇腾芯片支持NPU卡内存发生UCE故障（内存不可修复）的实时检测，检测到UCE故障后，基于优化器状态副本机制并完成故障卡的在线修复并继续训练，减少训练损失。

1. 进程不退出，无需重新创建通信域、图编译等耗时流程
2. 无需重新调度训练节点，减少集群调度和管理层面耗时

二、执行流程：

- 捕获到到UCE异常，识别内存异常地址是否为权重地址，如果是，后续需要进行权重恢复
- 停止UCE故障设备，同时将正常设备置于等待状态
- 清理资源，包括运行时临时资源和状态
- 修复故障内存，如果是异常地址是权重地址，则通过冗余关系拷贝其他device上权重至故障设备上
- 训练数据集和训练状态（epoch、step等信息）回滚，恢复训练。

三、使用方式：通过环境变量export MS_ENABLE_TFT={UCE:1}使能框架能力
UCE: Uncorrectable Memory Error



网卡故障借轨回切

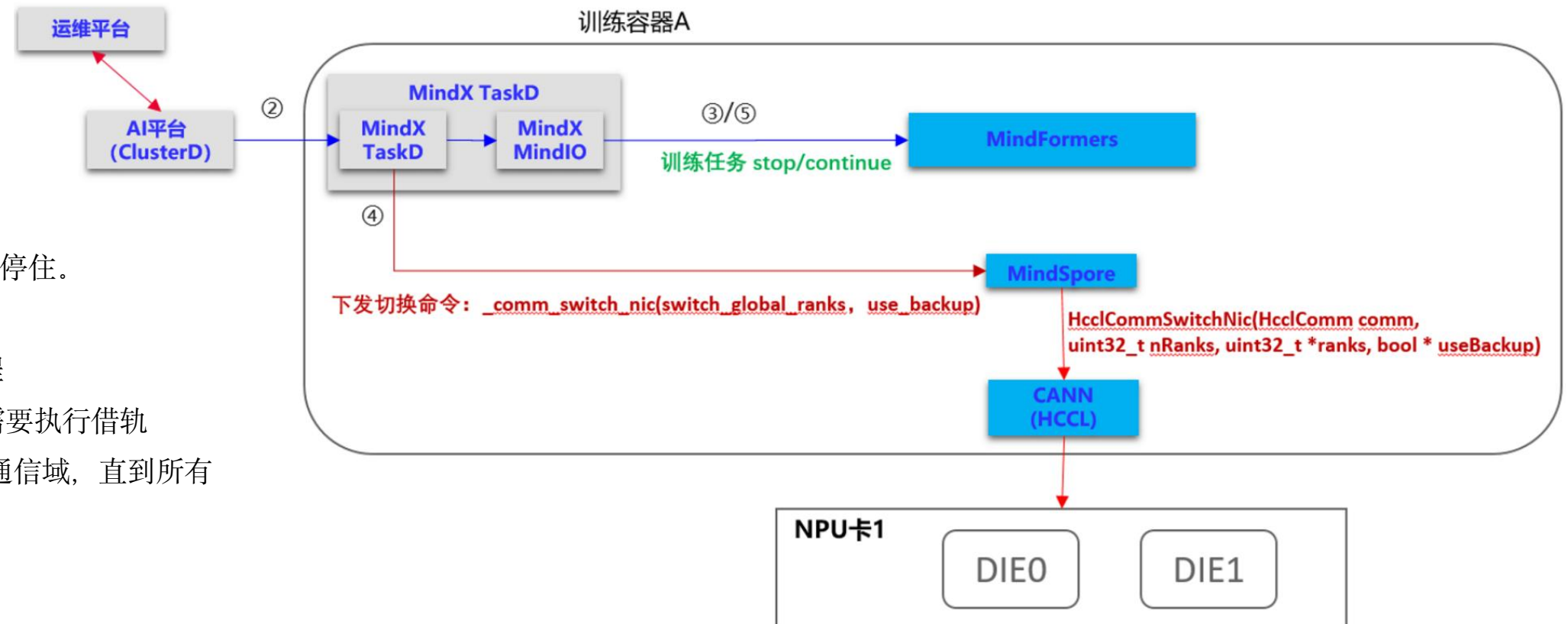
一、功能介绍：模型训练过程中，由于网卡故障实现网络通信主动借轨回切，训练业务不中断进程不退出，无需重新创建通信域、图编译等耗时流程

1. 进程不退出，无需重新创建通信域、图编译等耗时流程
2. 无需重新调度训练节点，减少集群调度和管理层面耗时

二、执行流程：

- 识别到网卡故障，在当前训练step结束后，通过callback将训练停住。
- 执行流同步，保证当前step所有异步下发的算子执行完毕
- 根据global rankid下发需要执行网卡借轨的任务到每个训练进程
- 训练进程轮训自己所属通信域以及对应rankid信息，判断是否需要执行借轨
- 如果需要执行则进行借轨回切动作，如果不需要，轮训下一个通信域，直到所有通信域遍历完毕
- 借轨结束，放开训练阻塞，继续训练

三、使用方式：通过环境变量export MS_ENABLE_TFT={TSP:1}使能框架能力
TSP: Training Step Pause



本章节以 Llama2-13B 训练为例演示临终 CKPT 的使用。

一、安装 MindSpore 和 MindIO

二、下载MindSporeTransformers，修改 configs/llama2/pretrain_llama2_13b_bf16.yaml 配置文件，主要配置如右。需要注意以下关键点：

- 1) sink_size: 1：临终 CKPT故障容错恢复等特性不支持 sink_size 大于 1 的场景，因此这里配置为 1。
- 2) enable_parallel_optimizer: True：使能优化器并行。
- 3) optimizer_weight_shard_size: 4：优化器并行的切分大小为 4。
- 4) data_parallel: 8: 数据并行配置为 8。

按照前面章节的说明，data_parallel/optimizer_weight_shard_size 的值为 $8 / 4 = 2$ ，大于 1，因此存在副本关系。

三、执行下面命令启动训练

```
export MINDIO_FOR_MINDSPORE=1

export MS_ENABLE_TFT="{TTP:1,UCE:1,ARF:1,TSP:1}"
export MS_TFT_IP=127.0.0.1
export MS_TFT_PORT=30051

bash scripts/msrun_launcher.sh "run_mindformer.py \
  --config configs/llama2/pretrain_llama2_13b_bf16.yaml \
  --train_dataset_dir "/YourDataSetPath" \
  --use_parallel True --run_mode train" 8
```

四、待训练执行若干个 step 之后，终止 worker 进程，触发临终 CKPT 保存

注意：通过上述启动方式，MindIO Controller 附着在 worker 0 进程上，此种情况下不能终止 worker 0，否则导致 MindIO Controller 退出，无法触发临终 CKPT。但是通过 taskd 方式启动训练时，MindIO Controller 是个单独的进程，可以终止 worker 0 进程。

```
# runner config
runner_config:
  epochs: 2
  batch_size: 4
  sink_mode: True
  sink_size: 1

# .....

# parallel context config
parallel:
  parallel_mode: 1 # 0-data parallel, 1-semi-auto parallel, 2-auto parallel, 3-hybrid parallel
  gradients_mean: False
  enable_alltoall: False
  full_batch: True
  search_mode: "sharding_propagation"
  enable_parallel_optimizer: True
  strategy_ckpt_save_file: "./ckpt_strategy.ckpt"
  parallel_optimizer_config:
    gradient_accumulation_shard: False
    parallel_optimizer_threshold: 64
    optimizer_weight_shard_size: 4

# .....

# default parallel of device num = 16 for Atlas 800T A2
parallel_config:
  data_parallel: 8
  model_parallel: 1
  pipeline_stage: 1
  use_seq_parallel: False
  micro_batch_num: 1
  vocab_emb_dp: True
  gradient_aggregation_group: 4
```

五、确认临终的 CheckPoint 生成

在整个训练进程结束后，通过日志确认最终生成的 CheckPoint 文件的合理性，具体操作如下：

1). 执行命令 `find output/checkpoint/ -name '*.ckpt'` 查找生成的 CheckPoint 文件：

```
$ find output/checkpoint/ -name '*.ckpt'
output/checkpoint/rank_2/llama2_13b_rank_2-5_1.ckpt
output/checkpoint/rank_3/llama2_13b_rank_3-5_1.ckpt
output/checkpoint/rank_0/llama2_13b_rank_0-5_1.ckpt
output/checkpoint/rank_5/llama2_13b_rank_5-5_1.ckpt
```

2) 执行命令 `cat output/msrun_log/worker_0.log | grep 'Epoch:'` 查看已经训练的 step：

```
$ cat output/msrun_log/worker_0.log | grep 'Epoch:'
2025-04-07 15:34:27,308 - [mindformers/core/callback/callback.py:529] - INFO - { Epoch:[ 1/ 2], step:[ 1/ 19], loss: 10.649, per_step_time: 10332
2025-04-07 15:34:29,173 - [mindformers/core/callback/callback.py:529] - INFO - { Epoch:[ 1/ 2], step:[ 2/ 19], loss: 10.633, per_step_time: 1752m
2025-04-07 15:34:30,941 - [mindformers/core/callback/callback.py:529] - INFO - { Epoch:[ 1/ 2], step:[ 3/ 19], loss: 9.673, per_step_time: 1754ms
2025-04-07 15:34:32,704 - [mindformers/core/callback/callback.py:529] - INFO - { Epoch:[ 1/ 2], step:[ 4/ 19], loss: 9.287, per_step_time: 1756ms
2025-04-07 15:34:34,469 - [mindformers/core/callback/callback.py:529] - INFO - { Epoch:[ 1/ 2], step:[ 5/ 19], loss: 8.867, per_step_time: 1758ms
```

3). 执行命令 `cat output/msrun_log/worker_0.log | grep 'report group list:'` 查看日志中 MindIO 输出的副本关系：

```
$ cat output/msrun_log/worker_0.log | grep 'report group list:'
2025-04-07 15:34:27.363613 info 1879138 [TTP controller.cpp:1512] rank:4, report group list: [0, 4]
2025-04-07 15:34:27.385564 info 1879139 [TTP controller.cpp:1512] rank:7, report group list: [3, 7]
2025-04-07 15:34:27.393198 info 1879136 [TTP controller.cpp:1512] rank:6, report group list: [2, 6]
2025-04-07 15:34:27.393515 info 1879142 [TTP controller.cpp:1512] rank:1, report group list: [1, 5]
```

从上面训练的 step 信息可以看出已经训练的 5 个 step，和 CheckPoint 的文件名 llama2_13b_rank_2-5_1.ckpt 中的 5 是一致的。

从日志中输出的副本关系 [0, 4]、[3, 7]、[2, 6] 和 [1, 5] 得知：

rank 0 和 rank 4 权重存在副本关系，临终的 Checkpoint 保存在 rank 0

rank 3 和 rank 7 权重存在副本关系，临终的 Checkpoint 保存在 rank 3

rank 2 和 rank 6 权重存在副本关系，临终的 Checkpoint 保存在 rank 2

rank 1 和 rank 5 权重存在副本关系，由于 worker 1 终止，临终的 Checkpoint 保存在 rank 5

谢谢观看！